26-1 이니로
알고리즘 멘토링

멘토 - 김수성

2주차 – Union Find

Union Find

Union Find

Union 연산과 Find 연산을 지원하는 자료구조

Union

두 원소가 속한 집합을 하나의 집합으로 합침

Find

어떤 원소가 어떤 집합에 속해 있는 지 구함

Union Find

해외에서는 Disjoint Set Union 이라고 부름

Disjoint Set

-> 서로소 집합

서로소

두 정수 A, B에 대해서 gcd(a, b) = 1

서로소 집합

두 집합 A, B에 대해서 $A \cap B = \emptyset$

Union Find

서로소 집합

두 집합 A, B에 대해서 $A \cap B = \emptyset$

Union

두 원소가 속한 집합을 하나로 합침

Find

어떤 원소가 어떤 집합에 속해 있는 지 구함

각 집합은 서로소

-> 무조건 한 원소는 한 집합에만 속할 수 있음

집합의 표현 / 1717

백준 1717 / https://www.acmicpc.net/problem/1717

0부터 N까지 총 N+1개의 원소가 있고, (N <= 1e6)
처음에는 각 원소가 자기 자신만 있는 집합에 속함

M개의 연산이 주어지고, (M <= 1e5)
연산 0 a b는 a가 속한 집합과 b가 속한 집합을 합침

연산 1 a b는 a와 b가 같은 집합에 속해 있는지 확인,
확인 연산의 결과가 같으면 YES, 다르면 NO를 출력

집합의 표현 / 1717

연산 0 a b는 a가 속한 집합과 b가 속한 집합을 합침
-> Union 연산

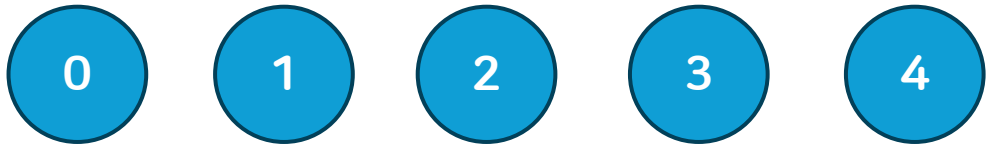
연산 1 a b는 a와 b가 같은 집합에 속해 있는지 확인
-> Find 연산

집합의 표현 / 1717

연산 0 a b는 a가 속한 집합과 b가 속한 집합을 합침
-> Union 연산

Union 연산을 구현하기 위해서 자료구조를
그래프로 관리해보자

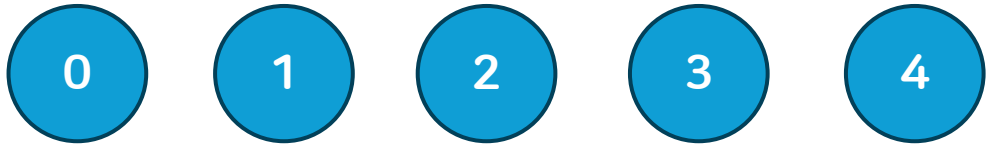
서로 같은 연결 요소면 같은 집합,
아니면 다른 집합으로 표현 할 수 있다



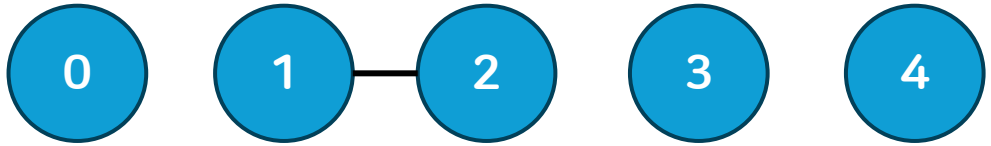
집합의 표현 / 1717

서로 같은 연결 요소면 같은 집합,
아니면 다른 집합으로 표현 할 수 있다

Union 연산은 두 원소를 같은 집합으로 합치는 과정
-> 서로 다른 원소에 대해서 간선을 연결하면 됨

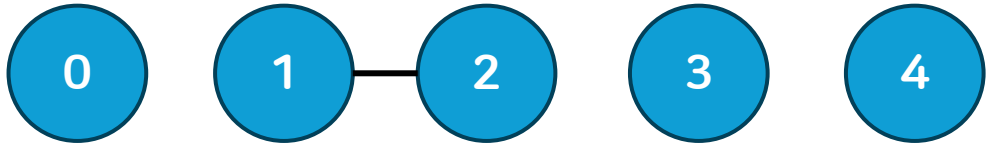


Union (1, 2)

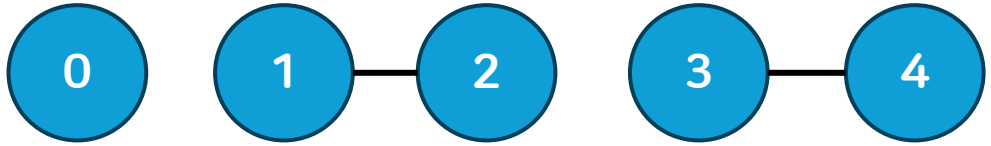


집합의 표현 / 1717

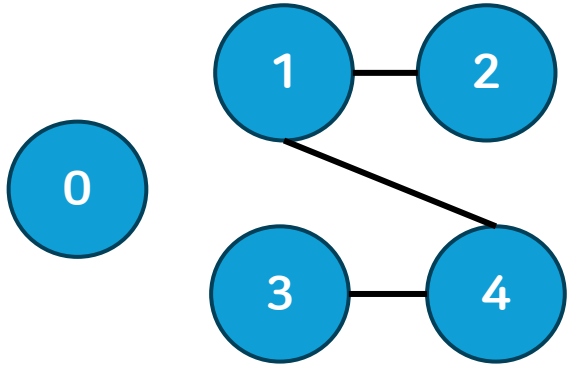
Union 연산은 다른 집합을 같은 집합으로 합치는 과정
-> 서로 다른 집합에 대해서 간선을 연결하면 됨



Union (3, 4)



Union (1, 4)

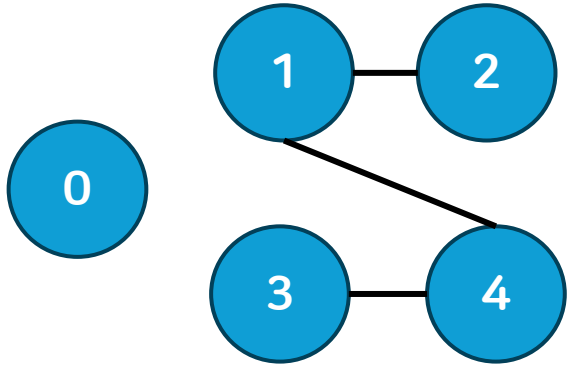


집합의 표현 / 1717

Union 연산은 다른 집합을 같은 집합으로 합치는 과정
-> 서로 다른 집합의 원소에 대해서 간선을 연결하면 됨

이미 같은 집합에 속한 원소들은 합쳐 줄 필요가 없음
-> 사이클이 생기지 않음

사이클이 없는 것은 각 집합은 트리인 것과 동치
-> 전체 자료구조는 포레스트



집합의 표현 / 1717

사이클이 없는 것은 각 집합은 트리인 것과 동치
-> 전체 자료구조는 포레스트

각 집합은 트리이기 때문에 사실 간선을 관리할 필요가 없음

트리의 성질
-> 루트가 아닌 노드는 부모를 1개 가짐

집합의 표현 / 1717

트리의 성질

-> 루트가 아닌 노드는 부모를 1개 가짐

Union 연산

서로 다른 집합의 원소에 대해서 간선을 연결하는 대신
한 집합의 원소의 부모를 다른 집합의 원소로 설정

하지만 루트가 아니면 이미 부모가 있음

-> 루트를 빨리 찾는 연산?

루트를 빨리 찾기만 하면 부모 설정만 하면 되므로
시간 복잡도는 루트를 찾는 시간에 달려있음

집합의 표현 / 1717

연산 1 a b는 a와 b가 같은 집합에 속해 있는지 확인
-> Find 연산

a가 속한 집합을 돌면서 b를 찾는 것은 비효율적임

+ Union 연산을 트리 구조로 구성했으므로
부모의 정보만 들고 있으므로 순회를 할 수 없음

각 원소는 한 집합에만 속하기 때문에
원소에 대한 대푯값을 관리하고 그 대푯값을 비교하면
Find 연산을 구현 할 수 있음

집합의 표현 / 1717

각 원소는 한 집합에만 속하기 때문에
원소에 대한 대푯값을 관리하고 그 대푯값을 비교하면
Find 연산을 구현 할 수 있음

대푯값을 어떤 노드로 설정 하는 게 좋을까?

트리 형태로 자료구조를 다루기 때문에
자연스럽게 루트 노드를 대푯값으로 설정

+ Union 연산에서 한 집합의 루트의 부모를
다른 집합의 원소로 설정해야 하기 때문에 타당해 보임

집합의 표현 / 1717

트리 형태로 자료구조를 다루기 때문에
자연스럽게 루트 노드를 대푯값으로 설정

Find 연산
-> 루트 노드를 만날 때 까지 부모를 타고 올라가면 됨

Union 연산
-> a에 대해서 Find 연산을 수행 후에
p[Find(a)] = b 로 설정하면 됨

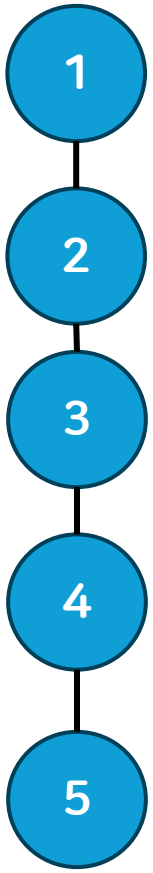
Find 연산 -> O(depth(tree))
Union 연산 -> Find 연산과 동일 O(depth(tree))

집합의 표현 / 1717

Find 연산 -> O(depth(tree))
Union 연산 -> Find 연산과 동일 O(depth(tree))

트리의 깊이만큼 비용이 발생함
다음과 같은 트리가 만들어진다면?

최악의 경우에 O(N)의 시간이 걸림

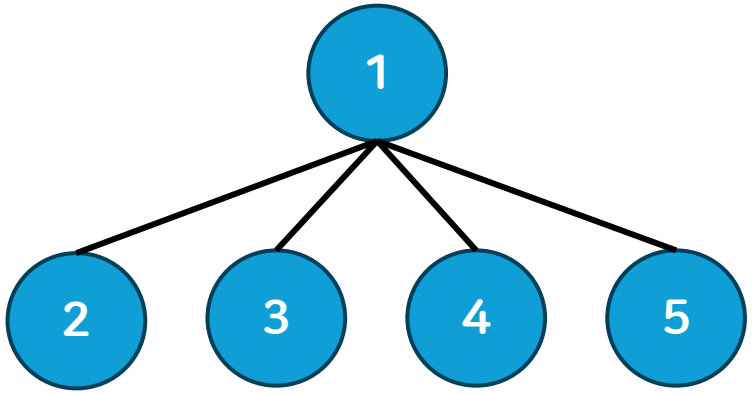


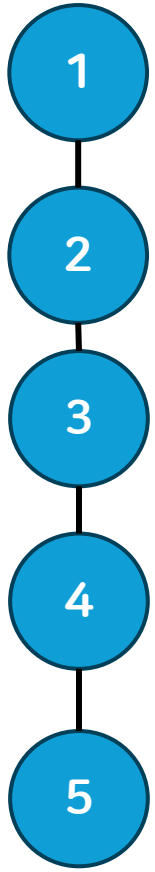
집합의 표현 / 1717

최악의 경우에 O(N)의 시간이 걸림

자료구조의 전제 조건
-> 각 집합은 트리, 대푯값은 루트 노드

위의 전제 조건을 지키면서 트리의 높이를 줄여보자





집합의 표현 / 1717

자료구조의 전제 조건
-> 각 집합은 트리, 대푯값은 루트 노드

위의 전제 조건을 지키면서 트리의 높이를 줄여보자

대푯값이 중요하기 때문에 Union 연산을
다음처럼 바꿔도 결과는 같음

p[Find(a)] = b -> p[Find(a)] = Find(b)
한 트리를 다른 트리의 루트 아래에 바로 붙이면
트리의 높이가 줄어듬

집합의 표현 / 1717

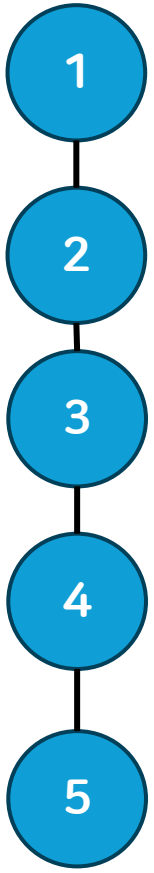
Path Compression

Find 연산 시에 경로를 압축하는 테크닉

Find 연산은 부모를 타고 올라가기 때문에
재귀 함수 형태로 나타낼 수 있음

```
int find(int x){
    if(p[x] == -1) return x;
    return find(p[x]);
}
```

Find(1) = Find(2) = Find(3) = Find(4) = Find(5) = 1

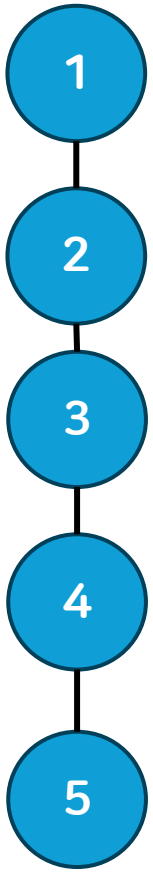


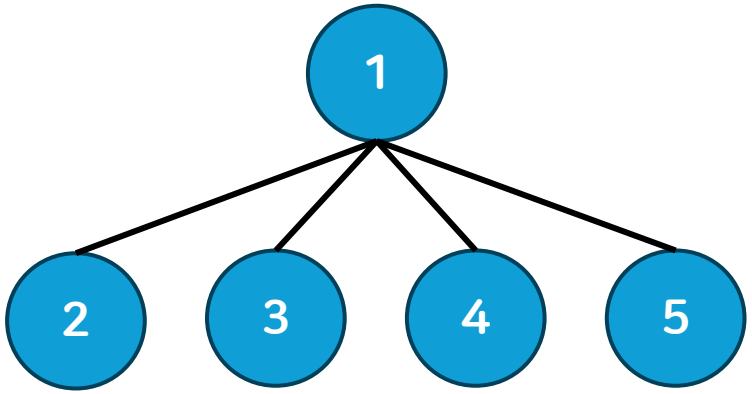
집합의 표현 / 1717

```
int find(int x){
    if(p[x] == -1) return x;
    return find(p[x]);
}
```

```
int find(int x){
    if(p[x] == -1) return x;
    p[x] = find(p[x]);
    return p[x];
}
```

오른쪽의 코드처럼 재귀 함수에서 부모를
루트 노드로 바꿔주는 과정을 넣으면
Find 연산 후에는 트리가 다음처럼 바뀌게 됨





집합의 표현 / 1717

Union by Size

두 원소를 Union 할 때

원소의 개수가 많은 집합을 부모로 설정하는 테크닉

```
bool merge(int a, int b){
    a = find(a);
    b = find(b);
    if(a == b) return false;
    p[a] = b;
    return true;
}
```

```
bool merge(int a, int b){
    a = find(a);
    b = find(b);
    if(a == b) return false;
    if(size[a] < size[b]) std::swap(a, b);

    p[b] = a;
    size[a] += size[b];

    return true;
}
```

집합의 표현 / 1717

두 원소를 Union 할 때
원소의 개수가 많은 집합을 부모로 설정하는 테크닉

항상 작은 집합이 큰 집합의 아래로 들어 감

작은 집합이 큰 집합보다 깊이가 큼
-> 최대 깊이는 1 증가

큰 집합이 작은 집합보다 깊이가 큼
-> 최대 깊이는 동일

집합의 표현 / 1717

항상 작은 집합이 큰 집합의 아래로 들어 감

작은 집합이 큰 집합보다 깊이가 큼
-> 최대 깊이는 1 증가

작은 집합의 크기 A, 큰 집합의 크기 B / 즉 B >= A
-> A + B >= 2 * A

즉 작은 집합의 크기는 항상 2배로 커짐
전체 집합의 크기는 N이므로 깊이 1 증가하는 시점은 최대 log(N)

Union, Find 연산의 시간 복잡도 -> log(N)

집합의 표현 / 1717

두 테크닉을 동시에 적용하면 시간 복잡도는 다음과 같음

amortized O(α(N))

N이 10^80 정도 되어도 4.. 정도 나옴
-> 상수 시간이라고 생각해도 무방 함

질문?

닭싸움 팀 정하기 / 1765

백준 1765 / https://www.acmicpc.net/problem/1765

학생들 사이에는 친구(F) 관계와 원수(E) 관계가 존재 함
팀은 다음 규칙을 만족해야 함
친구의 친구는 친구
원수의 원수도 친구

두 학생이 친구이면 같은 팀에 속해 있어야 하고,
주어진 인간관계를 바탕으로,
만들 수 있는 **팀의 최대 개수를 구해야 함**

닭싸움 팀 정하기 / 1765

F 쿼리 -> 친구의 친구는 친구

같은 팀을 하나의 집합으로 관리하자

두 학생이 친구이면 같은 팀에 속해 있어야 하고,
친구의 친구는 친구이기 때문에

F 쿼리가 들어오면 두 인원은 같은 팀
즉, 같은 집합에 넣어줘야 한다

```
char op; int a, b; cin >> op >> a >> b;
if(op == 'F'){
    merge(a, b);
}
```

-> Union Find로 관리 할 수 있음

닭싸움 팀 정하기 / 1765

E 쿼리 -> 원수의 원수는 친구

각 인원의 원수 목록을 관리를 해보자

```
else{
    arr[a].push_back(b);
    arr[b].push_back(a);
}
```

어떤 인원의 원수들은 서로 친구이기 때문에
그 인원의 원수들끼리 같은 집합으로 묶어주면 된다

Union Find는 같은 연결 요소면
같은 집합이기 때문에 대표 값 하나를
설정해서 그 원소랑 merge를 해주면 됨

```
for(int i = 1;i <= n;i++){
    if(arr[i].empty()) continue;
    for(int j = 1;j < arr[i].size();j++){
        merge(arr[i][0], arr[i][j]);
    }
}
```

닭싸움 팀 정하기 / 1765

이제 팀의 개수를 세주면 정답을 구할 수 있음

find(i)는 i가 속한 집합의 루트를 반환하고
루트는 한 집합에 한 개이기 때문에
루트의 개수를 세주면 집합의 개수를 셀 수 있음

```
for(int i = 1;i <= n;i++){
    if(find(i) != i) continue;
    result++;
}
```

닭싸움 팀 정하기 / 1765

시간복잡도

쿼리 처리 O(M * α(N))

Union Find 초기화, 집합 개수 세기 O(N)

-> O(N + M * α(N))

질문?

공항 / 10775

백준 10775 / https://www.acmicpc.net/problem/10775

공항에 G개의 게이트가 있고, P개의 비행기가 도착할 예정이다

i번째 비행기부터 게이트 중 하나에 도킹하고,
각 비행기는 1 ~ g[i]의 게이트에 도킹 할 수 있을 때
최대로 도킹할 수 있는 비행기의 수를 구해야 함

공항 / 10775

i번째 비행기부터 게이트 중 하나에 도킹하고,
각 비행기는 1 ~ g[i]의 게이트에 도킹 할 수 있을 때
최대로 도킹할 수 있는 비행기의 수를 구해야 함

우선 각 비행기는 1 ~ g[i]의 게이트

즉, 앞쪽에 배치를 할 수 있기 때문에
최대한 뒤에 배치를 하는 그리디가 항상 최적

-> 선택지가 적은 비행기들의 선택 폭이 늘어나기 때문

공항 / 10775

최대한 뒤에 배치를 하는 그리디가 항상 최적

각 비행기 순서대로 번호가 g[i] 이하인 아직 사용하지 않은 게이트 중 최대값을 찾고 그 곳에 비행기를 할당해야 함

-> 어떤 수 보다 작은 수 중 제일 큰 수를 빠르게 찾아야 함

공항 / 10775

어떤 수 보다 작은 수 중 제일 큰 수를 빠르게 찾아야 함

find(x) = 현재 사용 가능한 게이트 중 x 이하에서 가장 큰 번호

초기에는 다 사용 가능 하므로
find(1) = 1
find(2) = 2
...
find(G) = G

공항 / 10775

find(x) = 현재 사용 가능한 게이트 중 x 이하에서 가장 큰 번호

이제 4번 게이트를 사용했다고 생각해보자
find(4) = 4 -> 3
4번 게이트를 사용 했으므로 find(4)의 값은 3이 되어야 함

즉 merge(3, 4)를 해줘야 함

공항 / 10775

find(x) = 현재 사용 가능한 게이트 중 x 이하에서 가장 큰 번호

이미 2, 3번 게이트를 사용했고
이번에 4번 게이트를 사용한다고 생각해보자
find(1) = find(2) = find(3) = 1

find(4) = 4 -> 1

merge(1, 4) = merge(3, 4)
즉 x번 게이트를 사용하면 merge(x – 1, x) 를 해주면 됨

공항 / 10775

```
int p[MAX], sz[MAX];
bool use[MAX];

int find(int x){
    if(p[x] == -1) return x;
    return p[x] = find(p[x]);
}

void merge(int a, int b){
    a = find(a); b = find(b);
    if(a == b) return;
    p[b] = a;
}

void run(){
    cin >> n >> m; memset(p, -1, sizeof(p));
    for(int i = 1;i <= n;i++) sz[i] = 1;

    int result = 0;
    for(int i = 1;i <= m;i++){
        int v; cin >> v;
        int mx = find(v);
        if(use[mx]) break; use[mx] = 1;
        if(mx != 1) merge(mx - 1, mx);
        result++;
    }

    cout << result;
}
```

공항 / 10775

시간 복잡도

Union Find 초기화 O(G)

쿼리 O(P * α(G))

-> O(G + P * α(G))

질문?

트리 / 13306

백준 13306 / https://www.acmicpc.net/problem/13306

루트가 1인 트리가 주어지고, 두 종류의 쿼리가 주어짐
0 b: b와 부모를 잇는 간선을 제거
1 c d: 현재 시점에서 c와 d 사이에 경로가 존재하는지 판단

쿼리를 주어진 순서대로 처리하면서,
1번 쿼리에 대해 YES 또는 NO를 출력하면 됨

트리 / 13306

1 c d: 현재 시점에서 c와 d 사이에 경로가 존재하는지 판단

우선 각 정점이 연결되어 있으면 같은 집합으로 관리를 해보자
그러면 1번 쿼리는 서로 같은 집합에 속해 있는지 판단 해주면 됨

트리 / 13306

O b: b와 부모를 잇는 간선을 제거

기본적으로 유니온 파인드는 집합을 합치는 건 쉽지만
집합을 분리 하는 건 매우 어려움

-> 분리 연산을 병합 연산으로 만들 수는 없을까?

트리 / 13306

분리 연산을 병합 연산으로 만들 수는 없을까?

분리 연산을 뒤집으면 병합 연산
-> 쿼리를 거꾸로 처리해보자

쿼리를 뒤에서부터 처리하면서
0번 쿼리면 merge, 1번 쿼리면 find로 처리 해주면 풀 수 있음

또한 모든 0번 쿼리의 값은 다르고 개수가 N – 1개이기 때문에
모든 쿼리를 수행 했을 때는 모든 정점이 독립적이므로
따로 연결을 처리 해 줄 필요 없음

트리 / 13306

```
vector <tuple<int, int, int>> query;
vector <string> result;
void run(){
    cin >> n >> m; memset(p, -1, sizeof(p));
    for(int i = 2;i <= n;i++) cin >> pa[i];
    for(int i = 1;i <= n;i++) sz[i] = 1;

    int q = n + m - 1;
    while(q--){
        int op; cin >> op;
        if(!op){
            int x; cin >> x;
            query.push_back({op, x, pa[x]});
        }
        else{
            int a, b; cin >> a >> b;
            query.push_back({op, a, b});
        }
    }

    reverse(all(query));
    for(auto& [op, a, b] : query){
        if(!op) merge(a, b);
        else{
            result.push_back(find(a) == find(b) ? "YES" : "NO");
        }
    }

    reverse(all(result));
    for(auto& i : result) cout << i << "\n";
}
```

트리 / 13306

시간 복잡도

reverse O(N + M)

쿼리 O((N + M) * α(N))

-> O((N + M) * α(N))

질문?

고생하셨습니다